Algorithm: Spatial-channel collaborative multi-scale graph interaction deep transfer learning

Input:

- Source domains: S = { S_1, S_2,..., S_M }
- Target domain: T
- Batch size N_m: 64
- Number of categories: K (As the case changes, Case 1 is 4 and Case 2 is 7)
- Gate threshold: 0.6
- Group size: 2
- Scale level H: 7
- Gaussian kernel parameter: 0.5
- Pseudo-label preset threshold: 0.5

Output:

- Classification predictions for target domain samples

1: // ===== **3.1: Spatial-Channel Collaborative Prototype Extraction Module** =====

2: for each batch (X_src, y_src) from S and (X_tgt, y_tgt) from T do

3: // Spatial refinement

4: $X_{\text{out}} = \beta + \dfrac{\psi(X-\mu)}{\sqrt{\sigma^2+\varepsilon}}$ // Eq. (1)

5: γ = learnable_parameter in GN

6: $W_\gamma = \{w_i\} = \gamma_i / \left(\sum_{j=1}^{D}\gamma_j\right), \quad i,j = 1,2,\ldots,D$ // Eq. (2)

7: W = Gate(Sigmoid(W_γ))

8: W_1, W_2 = Gate_threshold (W)

10: $X^{w1} = X_{11}^{w} \oplus X_{22}^{w}$ // Eq. (3)

11: $X^{w2} = X_{21}^{w} \oplus X_{12}^{w}$ // Eq. (4)

12: X^{w} = Concat(X^{w1}, X^{w2}) // Refined spatial feature

13: // Channel refinement

14: (X_{up} , X_{low}) = Split(X^{w} , 0.5)

15: $Y_1 = X_{up}\left(\lambda^{GWC} + \lambda^{PWC_1}\right)$ - group size $g = 2$ // Eq. (5)

16: $Y_2 = \lambda^{PWC_2} X_{low} \cup X_{low}$ // Eq. (6)

17: O_1, O_2 = GlobalAveragePool(Y_1), GlobalAveragePool(Y_2)

18: I_1 = exp(O_1) / (exp(O_1) + exp(O_2)) // Eq. (7)

19: I_2 = exp(O_2) / (exp(O_1) + exp(O_2))

20: $Y = I_1 * Y_1 + I_2 * Y_2$ // Eq. (8) Refined channel feature

// Extract prototypes for source domains
for each source domain m = 1 to M do
    for each category k = 1 to K do
        $\hat{c}_k^{S_m}$ = mean(f_S (x_i)) for all x_i in category k // Eq. (9)
    end for
end for

// ===== Temporarily skip the target domain prototype update and proceed with graph construction and propagation =====
// Use the current target domain prototype (obtained from the previous batch update or initial value)
// If it is the first iteration, initialize to the source domain mean
if first_batch then
    for k = 1 to K do
        $\hat{c}_k^T$ = mean($\hat{c}_k^{S_m}$ for m=1,2,...,M)
    end for
end if

// ===== 3.2: Multi-Scale Graph Interaction Transfer Network =====
// Step 1: Build multi-domain knowledge graph

$$\mathbf{F} = \left[\underbrace{c_1^{S_1}\, c_2^{S_1} \cdots c_K^{S_1}}_{\text{prototypes of } S_1} \cdots \underbrace{c_1^{S_M}\, c_2^{S_M} \cdots c_K^{S_M}}_{\text{prototypes of } S_M} \; \underbrace{c_1^{T}\, c_2^{T} \cdots c_K^{T}}_{\text{prototypes of } T} \right]^{\mathrm{T}}$$

for i, j in 1 to $(M+1)K$ do
    $\mathbf{A}_{i,j} = G(\mathbf{F}_i^{\mathrm{T}}, \mathbf{F}_j^{\mathrm{T}})$ - Gaussian kernel parameter: 0.5 // Eq. (10)
end for

// Step 2: Build extended graph with query samples
for each query sample q in batch N_{m} do
    $\overline{\mathbf{F}} = \left[\mathbf{F}^{\mathrm{T}}\, f(q_1)\, f(q_2) \cdots f(q_{|N_m|}) \right]^{\mathrm{T}}$ // Eq. (11)
    $\mathbf{S}_{i,j} = G(\mathbf{F}_i^{\mathrm{T}}, f(q_j))$ // Eq. (12)
    $\overline{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{S} \\ \mathbf{S}^{\mathrm{T}} & \mathbf{I} \end{bmatrix}$ // Eq. (13)
end for

// Step 3: Multi-scale graph interaction (Scale level H =7)

X_list = [$\mathbf{X}^{(0)}$, $\mathbf{X}^{(1)}$, $\mathbf{X}^{(h)}$,…, $\mathbf{X}^{(H)}$]
for h = 1 to H do
    $\mathbf{A}\mathbf{X}^{(h-1)} \rightarrow \mathbf{X}^{(h)}$, $\forall h$=1, 2, ..., H // Eq. (14)
    X_list.append($\mathbf{X}^{(h)}$)
end for

// Dynamic interaction flow construction
for each node i = 1 to $|\overline{V}|$ do
    $\mathbf{Z}_i = \xi\left(\mathbf{X}_i^{(1)} \| \mathbf{X}_i^{(2)} \| \ldots \| \mathbf{X}_i^{(H)}\right)$ // Eq. (15)
    for each propagation step l do
        $\mathbf{X}_i^{(l)} = \mathbf{X}_i^{(l)} \| \mathbf{Z}_i$ // Eq. (16)
        $\tilde{w}_i(l) = Sig(\mathbf{X}_i^{(l)} \cdot s)$ // Eq. (17)
    end for
    // Normalize over scales
    for l = 1 to H do
        $w_i(l) = \dfrac{e^{\tilde{w}_i(l)}}{\sum_{h=0}^{H} e^{\tilde{w}_i(h)}}$ // Eq. (18)
    end for
end for

// Build weight matrix and fuse features
for h = 1 to H do
    $\eta_h[i] = w_i(h) \quad 1 \le i \le n$ for all nodes i // Eq. (19)
    $\mathbf{W}_h = Diag(\eta_h)$ // Eq. (20)
end for
$\mathbf{U}_\mathbf{X} = \sum_{h=0}^{H} \mathbf{W}_h \mathbf{X}^{(h)}$ // Eq. (21)

// Step 4: Cross-domain diagnosis
$\mathbf{P} = f_H(\overline{\mathbf{F}}, \overline{\mathbf{A}})$ // Eq. (22) - Classification probabilities
// Extract prototypes for target domain using confidence-thresholded pseudo-labeling
Now proceed with updating the target domain prototype (based on graph network output)
Extracting the classification probability of target samples from **P**
HC = empty_set // High confidence sample set
for j = 1 to N_target_samples do
    // Node index of the target sample in the extended graph
    node_idx = $|F|$ + j

p_t = **P**[node_idx] // The classification probability of the target sample

// Calculate entropy value

entropy_t = $-\sum_{k=1}^{K} p_t(k)\log p_t(k)$

if entropy_t < pseudo-label preset threshold then

pseudo_label = argmax(p_t)

// Store sample features and pseudo labels

HC.add

end if

end for

// Update target domain prototype

for each category k = 1 to K do

// Collect all high confidence sample features for this category

features_k = {z | (z, label) in HC and label = k}

if features_k not empty then

// Update the target domain prototype using high confidence samples

$\hat{c}_k^T$ = mean(features_k)

else

// If there are no high confidence samples, maintain the original value or use the source domain mean

// $\hat{c}_k^T$ remain unchanged

end if

end for

// ===== 3.3: Category Constraint Loss =====

// Global constraint loss

$$L_R^g = \frac{1}{(M+1)^4}\sum_{m,n,i,j=1}^{M+1} \| \mathbf{A}_{i,j} - \mathbf{A}_{m,n} \|_F \qquad \text{// Eq. (23)}$$

// Local constraint loss

$$L_R^l = \frac{1}{|N_m|}\sum_{k=1}^{K}\left(\sum_{(x_i^T, y_i^T)\in T_k} \| f(x_i^T) - c_k^T \|_2^2 + \sum_{m=1}^{M} \sum_{(x_i^{S_m}, y_i^{S_m})\in S_m^k} \| f(x_i^{S_m}) - c_k^{S_m} \|_2^2 \right) \qquad \text{// Eq. (24)}$$

// Classification losses

$$L_C^s = \frac{\sum_{m=1}^{M}\left(\mathbb{E}_{(x_i^{S_m}, y_i^{S_m}) \in \mathcal{S}_m} L_{ce}(p(x_i^{S_m}), y_i^{S_m})\right)}{M}$$ // Eq. (25)

$$L_C^p = \frac{\sum_{k=1}^{K} L_{ce}(p(c_k^T), k) + \sum_{m=1}^{M}\sum_{k=1}^{K} L_{ce}(p(c_k^{S_m}), k)}{K(M+1)}$$ // Eq. (26)

$$L_C^t = -\mathbb{E}_{(x_i^T, \hat{y}_i^T) \in T} \sum_{k=1}^{K} p(\hat{y}_i^T = k \mid x_i^T) \log p(\hat{y}_i^T = k \mid x_i^T)$$ // Eq. (27)

// Total loss

$$L_{CCL} = L_C + L_R,$$ // Eq. (28)

// Optimization

Update f_S, f_H, λ, s, γ, etc. via backpropagation

// Inference on target domain

for each target sample x_t in T do

    Compute extended graph with x_t

    $p_t = f_H(\overline{\mathbf{F}}, \overline{\mathbf{A}})$

    y_pred = argmax(p_t)

end for

return y_pred for all target samples